用户模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 User实体表

User类：对应用户表，包含用户基本信息属性和密码验证逻辑
【springboot/src/main/java/org/example/springboot/entity/User.java】

```
@Data
@TableName("user")
@Schema(description = "用户实体类")
public class User {
    @TableId(type = IdType.AUTO)
    @Schema(description = "用户ID")
    private Long id;

    @Schema(description = "用户名")
    @NotBlank(message = "用户名不能为空")
    @Size(min = 3, max = 50, message = "用户名长度必须在3到50个字符之间")
    private String username;

    @Schema(description = "密码")
    @NotBlank(message = "密码不能为空")
    @Size(min = 6, max = 100, message = "密码长度必须在6到100个字符之间")
    private String password;

    @Schema(description = "邮箱")
    @NotBlank(message = "邮箱不能为空")
    @Email(message = "邮箱格式不正确")
    private String email;

    @Schema(description = "手机号")
    @Pattern(regexp = "^1[3-9]\\d{9}$", message = "手机号格式不正确")
    private String phone;

    @Schema(description = "角色编码")
    private String roleCode;

    @Schema(description = "姓名")
    private String name;

    @Schema(description = "性别")
    private String sex;

    @Schema(description = "头像URL")
    private String avatar;

    @Schema(description = "状态(0:禁用,1:正常)")
    private Integer status;
```

半夜撕代码版权所有

```
    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    private String token;
}
```

1.2 核心属性说明

1. **用户认证属性**：
    - username：用户名，唯一标识，用于登录
    - password：密码，存储时使用BCrypt加密
    - token：非数据库字段，用于JWT认证
2. **用户基本信息**：
    - name：用户真实姓名
    - sex：性别
    - email：邮箱，用于通知和找回密码
    - phone：手机号，用于联系
    - avatar：头像URL，存储路径
3. **系统管理属性**：
    - roleCode：角色编码，如ADMIN、DOCTOR、NURSE、USER等
    - status：账户状态，使用AccountStatus枚举类管理
    - createTime：创建时间
    - updateTime：更新时间

1.3 关联数据传输对象

1.3.1 UserRegisterDTO

【springboot/src/main/java/org/example/springboot/DTO/UserRegisterDTO.java】

```
@Data
@Schema(description = "用户注册DTO")
public class UserRegisterDTO {
    @Schema(description = "用户名")
    @NotBlank(message = "用户名不能为空")
    @Size(min = 3, max = 50, message = "用户名长度必须在3到50个字符之间")
    private String username;

    @Schema(description = "密码")
    @NotBlank(message = "密码不能为空")
    @Size(min = 6, max = 100, message = "密码长度必须在6到100个字符之间")
    private String password;

    @Schema(description = "确认密码")
```

半夜撕代码版权所有

```java
    @NotBlank(message = "确认密码不能为空")
    private String confirmPassword;

    @Schema(description = "邮箱")
    @NotBlank(message = "邮箱不能为空")
    @Email(message = "邮箱格式不正确")
    private String email;

    @Schema(description = "手机号")
    @Pattern(regexp = "^1[3-9]\\d{9}$", message = "手机号格式不正确")
    private String phone;

    @Schema(description = "姓名")
    private String name;

    @Schema(description = "性别")
    private String sex;

    @Schema(description = "角色编码")
    private String roleCode = "USER"; // 默认为普通用户
}
```

1.3.2 UserPasswordUpdateDTO

【springboot/src/main/java/org/example/springboot/DTO/UserPasswordUpdateDTO.java】

```java
@Data
@Schema(description = "更新密码DTO")
public class UserPasswordUpdateDTO {
    @Schema(description = "旧密码")
    private String oldPassword;
    @Schema(description = "新密码")
    private String newPassword;
}
```

1.4 账户状态枚举类

【springboot/src/main/java/org/example/springboot/enumClass/AccountStatus.java】

```java
@Schema(description = "账户状态枚举")
public enum AccountStatus {
    @Schema(description = "禁用") DISABLED(0),
    @Schema(description = "启用") ENABLED(1),
    @Schema(description = "待审核") PENDING_REVIEW(2),
    @Schema(description = "审核通过") REVIEW_SUCCESS(3),
    @Schema(description = "审核不通过") REVIEW_FAILED(4);

    private final int value;

    AccountStatus(int value) {
        this.value = value;
    }

    public int getValue() {
```

半夜撕代码版权所有

```
        return value;
    }
}
```

二、各功能详细讲解

2.1 用户登录功能

2.1.1 功能概述

用户登录功能通过验证用户名和密码实现身份认证，成功后生成JWT令牌并返回用户信息，支持多种角色登录（管理员、医生、护士、普通用户），同时会检查账户状态是否正常。

2.1.2 详细流程

1. **前端流程**：
   - 用户在登录页面（vue3/src/views/auth/Login.vue）输入用户名和密码
   - 点击登录按钮触发handleLogin()方法
   - 前端进行表单验证后，调用request.post("/user/login", loginForm)向后端发送登录请求
   - 接收后端返回的用户信息和token，存储在localStorage和Pinia状态管理中
   - 根据用户角色跳转到不同页面（管理员/医生/护士跳转到后台，普通用户跳转到前台）
2. **后端流程**：
   - UserController的login方法接收登录请求
   - 调用UserService的login方法进行用户验证
   - 验证用户存在性和状态
   - 使用BCryptPasswordEncoder验证密码
   - 生成JWT令牌
   - 返回用户信息和token给前端

2.1.3 关键代码讲解

1. **前端登录请求**
   【vue3/src/views/auth/Login.vue】

```
const handleLogin = () => {
  loginFormRef.value.validate(async valid => {
    if (valid) {
      loading.value = true
      try {
        // 统一使用用户登录接口
        const res = await request.post("/user/login", loginForm, {
          successMsg: "登录成功，欢迎回来",
          showDefaultMsg: true,
          onSuccess: async (data) => {
            // 处理记住我功能
            if (rememberMe.value) {
              localStorage.setItem('rememberMe', 'true')
              localStorage.setItem('rememberedUsername', loginForm.username)
            } else {
```

```
            localStorage.removeItem('rememberMe')
            localStorage.removeItem('rememberedUsername')
          }

          userStore.setUserInfo(data)

          // 获取角色相关信息（医生或患者详情）
          await userStore.fetchRoleInfo()

          // 根据返回的角色决定跳转路径
          if (['ADMIN', 'DOCTOR', 'NURSE'].includes(data.roleCode)) {
            // 医护人员登录，设置菜单
            userStore.setMenus(data.menuList)

            // 如果是医生，获取医生详情信息
            if (data.roleCode === 'DOCTOR') {
             await fetchDoctorInfo(data.id)
            }

            // 直接导航到后台仪表盘
            await router.isReady()
            router.push(route.query.redirect || '/back/dashboard')
          } else {
            // 患者登录，直接跳转到前台
            const redirect = route.query.redirect || '/'
            router.push(redirect)
          }
        }
      })
    } finally {
      loading.value = false
    }
  }
})
}
```

2. **后端登录处理**
【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "登录")
@PostMapping("/login")
public Result<?> login(@RequestBody User user) {
    User loginUser = userService.login(user);
    return Result.success(loginUser);
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public User login(User user) {
    User dbUser = getByUsername(user.getUsername());
    // 用户存在性检查已经在 getByUsername 中处理
    if (dbUser.getStatus() == AccountStatus.PENDING_REVIEW.getValue()) {
        throw new ServiceException("账号正在审核");
    }
```

半夜撕代码版权所有

```
    if (dbUser.getStatus() == AccountStatus.REVIEW_FAILED.getValue()) {
        throw new ServiceException("账号审核不通过，请修改个人信息");
    }
    if (dbUser.getStatus() == AccountStatus.DISABLED.getValue()) {
        throw new ServiceException("账号已被禁用，请联系管理员");
    }
    if (!bCryptPasswordEncoder.matches(user.getPassword(), dbUser.getPassword()))
{
        throw new ServiceException("用户名或密码错误");
    }

    String token = JwtTokenUtils.genToken(String.valueOf(dbUser.getId()),
dbUser.getPassword());
    dbUser.setToken(token);
    return dbUser;
}
```

3. **JWT令牌生成**
   【springboot/src/main/java/org/example/springboot/util/JwtTokenUtils.java】

```
public static String genToken(String userId, String sign) {
    return JWT.create()
        .withAudience(userId)
        .withExpiresAt(DateUtil.offsetHour(new Date(), 2))
        .sign(Algorithm.HMAC256(sign));
}
```

2.1.4 功能实现细节

1. **状态验证**：
   - 登录前检查用户状态，禁止被禁用、审核中或审核不通过的账户登录
   - 使用AccountStatus枚举类管理不同的账户状态
2. **密码验证**：
   - 使用Spring Security的BCryptPasswordEncoder进行密码匹配
   - 密码不直接比较，而是通过matches方法验证哈希值
3. **令牌生成**：
   - 使用JWT（JSON Web Token）生成认证令牌
   - 令牌包含用户ID作为受众（audience）
   - 设置令牌过期时间为2小时
   - 使用用户密码作为签名密钥，确保安全性
4. **角色处理**：
   - 根据用户角色（roleCode）决定登录后的跳转路径
   - 管理员、医生、护士跳转到后台管理系统
   - 普通用户跳转到前台页面

半夜撕代码版权所有

2.2 用户注册功能

2.2.1 功能概述

用户注册功能允许新用户创建账号，包括基本信息填写、表单验证、密码加密存储等流程，支持普通用户和患者两种注册方式，注册成功后自动分配默认角色。

2.2.2 详细流程

1. **前端流程**：
   - 用户在注册页面（vue3/src/views/auth/Register.vue）填写注册信息
   - 前端进行表单验证（用户名、密码、邮箱格式等）
   - 点击注册按钮触发handleRegister()方法
   - 调用request.post("/patient/register", registerData)向后端发送注册请求
   - 注册成功后跳转到登录页面
2. **后端流程**：
   - UserController的register方法接收注册请求
   - 验证两次密码是否一致
   - 调用UserService的register方法处理注册
   - 检查用户名和邮箱唯一性
   - 使用BCryptPasswordEncoder加密密码
   - 设置默认状态和头像
   - 保存用户信息到数据库
   - 生成JWT令牌
   - 返回用户信息给前端

2.2.3 关键代码讲解

1. **前端注册请求**
   【vue3/src/views/auth/Register.vue】

```
const handleRegister = () => {
  registerFormRef.value.validate(async valid => {
    if (valid) {
      loading.value = true
      try {
        // 同步用户名和真实姓名
        registerForm.user.name = registerForm.patient.name
        registerForm.user.sex = registerForm.patient.sex
        registerForm.user.phone = registerForm.patient.phone

        // 移除确认密码和协议同意，不需要传递给后端
        const { confirmPassword, agreement, ...registerData } = registerForm

        // 使用患者注册接口
        await request.post("/patient/register", registerData, {
          successMsg: "注册成功，欢迎加入康泰医疗",
          showDefaultMsg: true,
          onSuccess: () => {
```

```
            ElMessage({
              type: 'success',
              message: '您的账号已创建成功，正在跳转到登录页面...'
            })
            setTimeout(() => {
              router.push('/login')
            }, 2000)
          }
        })
      } finally {
        loading.value = false
      }
    }
  })
}
```

2. **后端注册处理**
【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "用户注册")
@PostMapping("/register")
public Result<?> register(@Validated @RequestBody UserRegisterDTO registerDTO) {
    // 验证两次密码是否一致
    if (!registerDTO.getPassword().equals(registerDTO.getConfirmPassword())) {
        return Result.error("两次输入的密码不一致");
    }

    User user = userService.register(registerDTO);
    return Result.success(user);
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
@Transactional
public User register(UserRegisterDTO registerDTO) {
    // 检查用户名是否存在
    if (userMapper.selectOne(
            new LambdaQueryWrapper<User>()
                .eq(User::getUsername, registerDTO.getUsername())
        ) != null) {
        throw new ServiceException("用户名已存在");
    }

    // 检查邮箱是否被使用
    if (userMapper.selectOne(
            new LambdaQueryWrapper<User>()
                .eq(User::getEmail, registerDTO.getEmail())
        ) != null) {
        throw new ServiceException("邮箱已被使用");
    }

    User user = new User();
    BeanUtils.copyProperties(registerDTO, user);
```

半夜撕代码版权所有

```
    // 加密密码
    user.setPassword(bCryptPasswordEncoder.encode(user.getPassword()));

    // 设置默认状态
    user.setStatus(AccountStatus.ENABLED.getValue());

    // 设置默认头像
    if (StringUtils.isBlank(user.getAvatar())) {
        user.setAvatar("/img/default_avatar.png");
    }

    if (userMapper.insert(user) <= 0) {
        throw new ServiceException("用户注册失败");
    }

    // 生成token
    String token = JwtTokenUtils.genToken(String.valueOf(user.getId()),
user.getPassword());
    user.setToken(token);

    return user;
}
```

2.2.4 功能实现细节

1. **表单验证**：
   - 前端使用Element Plus的表单验证规则
   - 后端使用Jakarta Validation注解进行参数校验
   - 验证用户名、密码、邮箱、手机号等字段的格式和长度
2. **唯一性检查**：
   - 检查用户名和邮箱是否已被使用
   - 使用MyBatis-Plus的LambdaQueryWrapper构建查询条件
3. **密码处理**：
   - 前端验证两次输入密码是否一致
   - 后端使用BCryptPasswordEncoder加密密码
   - 密码不以明文存储在数据库中
4. **默认设置**：
   - 设置默认账户状态为启用（ENABLED）
   - 设置默认头像路径
   - 设置默认角色（普通用户或患者）

半夜撕代码版权所有

2.3 密码管理功能

2.3.1 功能概述

密码管理功能包括密码修改和密码重置（忘记密码）两部分，允许用户在登录状态下修改密码或在忘记密码时通过邮箱重置密码，确保账户安全性。

2.3.2 详细流程

1. **密码修改流程**：
   - 用户在个人信息页面输入旧密码和新密码
   - 前端验证两次输入的新密码是否一致
   - 调用request.put("/user/password/{id}", passwordData)发送修改请求
   - 后端验证旧密码是否正确
   - 加密新密码并更新数据库
   - 返回修改结果
2. **忘记密码流程**：
   - 用户在忘记密码页面输入注册邮箱和新密码
   - 前端验证两次输入的新密码是否一致
   - 调用request.post("/user/forget", forgetForm)发送重置请求
   - 后端验证邮箱是否存在
   - 加密新密码并更新数据库
   - 返回重置结果

2.3.3 关键代码讲解

1. **密码修改前端**
   【vue3/src/views/backend/user/PersonInfo.vue】

```
const updatePassword = async () => {
  if (passwordForm.newPassword !== passwordForm.confirmPassword) {
    ElMessage.error('两次输入的新密码不一致')
    return
  }

  try {
    await request.put(`/user/password/${userInfo.id}`, {
      oldPassword: passwordForm.oldPassword,
      newPassword: passwordForm.newPassword
    }, {
      successMsg: '密码修改成功',
      onSuccess: () => {
        passwordDialogVisible.value = false
        resetPasswordForm()
      }
    })
  } catch (error) {
    console.error('密码修改失败:', error)
  }
```

半夜撕代码版权所有

```
}
```

2. **密码修改后端**
【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "密码修改")
@PutMapping("/password/{id}")
public Result<?> updatePassword(@PathVariable Long id, @RequestBody
UserPasswordUpdateDTO userPasswordUpdate) {
    // 密码修改失败会抛出异常
    userService.updatePassword(id, userPasswordUpdate);
    return Result.success("密码修改成功");
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public void updatePassword(Long id, UserPasswordUpdateDTO update) {
    User user = userMapper.selectById(id);
    if (user == null) {
        throw new ServiceException("用户不存在");
    }

    // 验证旧密码
    if (!bCryptPasswordEncoder.matches(update.getOldPassword(),
user.getPassword())) {
        throw new ServiceException("原密码错误");
    }

    // 更新新密码
    user.setPassword(bCryptPasswordEncoder.encode(update.getNewPassword()));
    if (userMapper.updateById(user) <= 0) {
        throw new ServiceException("密码修改失败");
    }
}
```

3. **忘记密码前端**
【vue3/src/views/auth/ForgetPassword.vue】

```
const handleForgetPassword = () => {
  formRef.value.validate(async valid => {
    if (valid) {
      loading.value = true
      try {
        activeStep.value = 2
        await request.post("/user/forget", {
          email: forgetForm.email,
          newPassword: forgetForm.newPassword
        }, {
          successMsg: "密码重置成功，请使用新密码登录",
          onSuccess: () => {
            activeStep.value = 3
            ElMessage({
              type: 'success',
              message: '密码已重置，2秒后将跳转到登录页面'
```

```
            })
            setTimeout(() => {
              router.push('/login')
            }, 2000)
          }
        })
      } finally {
        loading.value = false
      }
    }
  })
}
```

4. **忘记密码后端**
【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "忘记密码")
@PostMapping("/forget")
public Result<?> forgetPassword(@RequestBody Map<String, String> forgetForm) {
    // 密码重置失败会抛出异常
    userService.forgetPassword(forgetForm.get("email"),
forgetForm.get("newPassword"));
    return Result.success("密码重置成功");
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public void forgetPassword(String email, String newPassword) {
    if (StringUtils.isBlank(email)) {
        throw new ServiceException("邮箱不能为空");
    }

    if (StringUtils.isBlank(newPassword)) {
        throw new ServiceException("新密码不能为空");
    }

    // 验证密码长度
    if (newPassword.length() < 6 || newPassword.length() > 100) {
        throw new ServiceException("密码长度必须在6到100个字符之间");
    }

    User user = userMapper.selectOne(
        new LambdaQueryWrapper<User>()
            .eq(User::getEmail, email)
    );

    if (user == null) {
        throw new ServiceException("邮箱不存在，请检查后重试");
    }

    // 更新密码
    user.setPassword(bCryptPasswordEncoder.encode(newPassword));

    if (userMapper.updateById(user) <= 0) {
```

```
        throw new ServiceException("密码重置失败，请稍后再试");
    }
}
```

2.3.4 功能实现细节

1. **安全验证**：
    - 修改密码时必须验证原密码是否正确
    - 忘记密码时通过邮箱验证用户身份
    - 使用BCryptPasswordEncoder进行密码匹配和加密
2. **密码规则**：
    - 密码长度必须在6到100个字符之间
    - 前端验证两次输入的新密码是否一致
    - 后端再次验证密码长度是否符合要求
3. **异常处理**：
    - 使用ServiceException抛出具体的业务异常
    - 前端显示友好的错误提示信息
    - 全局异常处理器统一处理异常

2.4 用户管理功能

2.4.1 功能概述

用户管理功能提供系统管理员对用户进行增删改查、状态管理等操作的能力，包括用户列表展示、分页查询、条件筛选、新增用户、编辑用户信息、启用/禁用用户、删除用户等功能。

2.4.2 详细流程

1. **用户列表查询流程**：
    - 管理员进入用户管理页面（vue3/src/views/backend/user/UserList.vue）
    - 页面加载时调用fetchUsers()方法获取用户列表
    - 前端发送request.get('/user/page')请求，携带分页和筛选参数
    - 后端UserController的getUsersByPage方法处理请求
    - 调用UserService的getUsersByPage方法构建查询条件
    - 使用MyBatis-Plus的Page对象进行分页查询
    - 返回分页数据给前端
    - 前端展示用户列表和分页信息
2. **新增用户流程**：
    - 管理员点击"新增用户"按钮，弹出用户表单对话框
    - 填写用户信息并点击确定按钮
    - 调用request.post('/user/add', form)发送创建请求
    - 后端UserController的createUser方法处理请求
    - 调用UserService的createUser方法进行用户创建
    - 检查用户名和邮箱唯一性

半夜撕代码版权所有

- 加密密码并保存用户信息
- 返回创建结果
- 前端刷新用户列表

3. **编辑用户流程**：
   - 管理员点击用户列表中的"编辑"按钮，弹出编辑表单
   - 修改用户信息并点击确定按钮
   - 调用request.put(`/user/${form.id}`, form)发送更新请求
   - 后端UserController的updateUser方法处理请求
   - 调用UserService的updateUser方法进行用户更新
   - 检查用户是否存在和用户名唯一性
   - 更新用户信息
   - 返回更新结果
   - 前端刷新用户列表
4. **用户状态管理流程**：
   - 管理员点击用户列表中的"启用"或"禁用"按钮
   - 调用request.put(`/user/status/${userId}?status=${status}`)发送状态更新请求
   - 后端UserController的updateStatus方法处理请求
   - 更新用户状态
   - 返回更新结果
   - 前端刷新用户列表
5. **删除用户流程**：
   - 管理员点击用户列表中的"删除"按钮，弹出确认对话框
   - 确认删除后调用request.delete(`/user/delete/${row.id}`)发送删除请求
   - 后端UserController的deleteUserById方法处理请求
   - 调用UserService的deleteUserById方法进行用户删除
   - 检查用户是否存在和是否被关联（医生或患者）
   - 执行删除操作
   - 返回删除结果
   - 前端刷新用户列表

2.4.3 关键代码讲解

1. **用户列表查询**
   【vue3/src/views/backend/user/UserList.vue】

```
// 获取用户列表
const fetchUsers = async () => {
  loading.value = true
  try {
    await request.get('/user/page', {
      username: searchForm.username,
      name: searchForm.name,
      roleCode: searchForm.roleCode,
```

半夜撕代码版权所有

```
      currentPage: currentPage.value,
      size: pageSize.value
    },{
      onSuccess: (res) => {
        tableData.value = res.records
        total.value = res.total
      }
    })
  } catch (error) {
    console.error('获取用户列表失败:', error)
  } finally {
    loading.value = false
  }
}
```

【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "分页查询用户")
@GetMapping("/page")
public Result<?> getUsersByPage(
        @RequestParam(defaultValue = "") String username,
        @RequestParam(defaultValue = "") String name,
        @RequestParam(defaultValue = "") String roleCode,
        @RequestParam(defaultValue = "1") Integer currentPage,
        @RequestParam(defaultValue = "10") Integer size) {
    Page<User> page = userService.getUsersByPage(username, name, roleCode,
currentPage, size);
    return Result.success(page);
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public Page<User> getUsersByPage(String username, String name, String roleCode,
Integer currentPage, Integer size) {
    LambdaQueryWrapper<User> queryWrapper = new LambdaQueryWrapper<>();

    // 添加查询条件
    if (StringUtils.isNotBlank(username)) {
        queryWrapper.like(User::getUsername, username);
    }
    if (StringUtils.isNotBlank(name)) {
        queryWrapper.like(User::getName, name);
    }
    if (StringUtils.isNotBlank(roleCode)) {
        queryWrapper.eq(User::getRoleCode, roleCode);
    }

    return userMapper.selectPage(new Page<>(currentPage, size), queryWrapper);
}
```

2. **新增用户**
   【vue3/src/views/backend/user/UserList.vue】

```
// 提交表单
```

半夜撕代码版权所有

```
const submitForm = () => {
  formRef.value.validate(async (valid) => {
    if (valid) {
      submitLoading.value = true
      try {
        // 新增用户
        if (dialogType.value === 'add') {
          await request.post('/user/add', form, {
            successMsg: '添加用户成功',
            onSuccess: () => {
              dialogVisible.value = false
              fetchUsers()
            }
          })
        } else {
          // 编辑用户
          await request.put(`/user/${form.id}`, form, {
            successMsg: '更新用户成功',
            onSuccess: () => {
              dialogVisible.value = false
              fetchUsers()
            }
          })
        }
      } finally {
        submitLoading.value = false
      }
    }
  })
}
```

【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "创建新用户")
@PostMapping("/add")
public Result<?> createUser(@RequestBody User user) {
    userService.createUser(user);
    return Result.success("创建成功");
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public void createUser(User user) {
    // 检查用户名是否存在
    if (userMapper.selectOne(
            new LambdaQueryWrapper<User>()
                .eq(User::getUsername, user.getUsername())
        ) != null) {
        throw new ServiceException("用户名已存在");
    }

    // 检查邮箱是否被使用
    if (userMapper.selectOne(
            new LambdaQueryWrapper<User>()
```

半夜撕代码版权所有

```
                .eq(User::getEmail, user.getEmail())
        ) != null) {
        throw new ServiceException("邮箱已被使用");
    }

    user.setPassword(StringUtils.isNotBlank(user.getPassword()) ?
user.getPassword() : DEFAULT_PWD);
    user.setPassword(bCryptPasswordEncoder.encode(user.getPassword()));

    if (userMapper.insert(user) <= 0) {
        throw new ServiceException("用户创建失败");
    }
}
```

3. **删除用户**

【vue3/src/views/backend/user/UserList.vue】

```
// 删除用户
const handleDelete = (row) => {
  ElMessageBox.confirm(
    `确定要删除用户"${row.username}"吗？此操作不可恢复。`,
    '警告',
    {
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'warning'
    }
  )
    .then(async () => {
      try {
        await request.delete(`/user/delete/${row.id}`, {
          successMsg: '删除成功',
          onSuccess: () => {
            fetchUsers()
          }
        })
      } catch (error) {
        console.error('删除用户失败:', error)
      }
    })
    .catch(() => {
      ElMessage({
        type: 'info',
        message: '已取消删除'
      })
    })
}
```

【springboot/src/main/java/org/example/springboot/controller/UserController.java】

半夜撕代码版权所有

```
@Operation(summary = "根据id删除用户")
@DeleteMapping("/delete/{id}")
public Result<?> deleteUserById(@PathVariable Long id) {
    // 删除失败会抛出异常
    userService.deleteUserById(id);
    return Result.success("删除成功");
}
```

【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
@Transactional
public void deleteUserById(Long id) {
    // 检查用户是否存在
    User user = userMapper.selectById(id);
    if (user == null) {
        throw new ServiceException("用户不存在");
    }

    // 检查用户是否被医生关联
    Doctor doctor = doctorMapper.selectOne(
        new LambdaQueryWrapper<Doctor>()
            .eq(Doctor::getUserId, id)
    );
    if (doctor != null) {
        throw new ServiceException("该用户已关联医生信息（" + doctor.getName() +
"），不能删除");
    }

    // 检查用户是否被患者关联
    Patient patient = patientMapper.selectOne(
        new LambdaQueryWrapper<Patient>()
            .eq(Patient::getUserId, id)
    );
    if (patient != null) {
        throw new ServiceException("该用户已关联患者信息（" + patient.getName() +
"），不能删除");
    }

    // 执行删除操作
    if (userMapper.deleteById(id) <= 0) {
        throw new ServiceException("删除失败");
    }
}
```

2.4.4 功能实现细节

1. **分页查询**：
    - 使用MyBatis-Plus的Page对象进行分页
    - 支持多条件组合查询（用户名、姓名、角色等）
    - 使用LambdaQueryWrapper构建动态SQL条件
2. **数据验证**：
    - 前端使用Element Plus的表单验证

半夜撕代码版权所有

    - 后端检查用户名和邮箱唯一性
    - 验证用户是否存在和是否可删除
3. **关联检查**：
    - 删除用户前检查是否被医生或患者关联
    - 防止误删除导致数据一致性问题
    - 提供明确的错误提示信息
4. **状态管理**：
    - 支持启用和禁用用户账号
    - 使用状态值控制用户的登录权限
    - 前端使用不同颜色的标签直观显示状态

2.5 当前用户信息管理

2.5.1 功能概述

当前用户信息管理功能允许已登录用户查看和修改自己的个人信息，包括基本信息更新、头像上传等，确保用户能够维护自己的最新信息。

2.5.2 详细流程

1. **获取当前用户信息流程**：
    - 前端通过Pinia状态管理获取本地存储的用户信息
    - 或调用request.get('/user/current')获取最新的用户信息
    - 后端UserController的getCurrentUser方法处理请求
    - 通过JwtTokenUtils.getCurrentUser()获取当前登录用户
    - 返回用户信息给前端
2. **更新用户信息流程**：
    - 用户在个人信息页面修改信息
    - 点击保存按钮提交更新
    - 调用request.put(`/user/${form.id}`, form)发送更新请求
    - 后端UserController的updateUser方法处理请求
    - 调用UserService的updateUser方法进行用户更新
    - 更新用户信息并返回结果
    - 前端更新本地存储的用户信息
3. **头像上传流程**：
    - 用户选择新头像图片
    - 前端调用customUploadAvatar方法处理上传
    - 创建FormData对象并添加文件
    - 发送request.post('/file/upload/img', formData)请求
    - 后端FileController处理文件上传
    - 保存文件并返回文件路径
    - 调用updateUserAvatar方法更新用户头像

半夜撕代码版权所有

- 前端更新用户信息显示

2.5.3 关键代码讲解

1. **获取当前用户信息**
   【springboot/src/main/java/org/example/springboot/controller/UserController.java】

```
@Operation(summary = "获取当前登录用户信息")
@GetMapping("/current")
public Result<?> getCurrentUser() {
    User currentUser = JwtTokenUtils.getCurrentUser();
    if (currentUser == null) {
        return Result.error("获取当前用户信息失败，请重新登录");
    }
    return Result.success(currentUser);
}
```

【springboot/src/main/java/org/example/springboot/util/JwtTokenUtils.java】

```
public static User getCurrentUser() {
    String token = null;
    try {
        HttpServletRequest request = ((ServletRequestAttributes)
RequestContextHolder.getRequestAttributes()).getRequest();
        token = request.getHeader("token");
        if (StringUtils.isBlank(token)) {
            token = request.getParameter("token");
        }
        if (StringUtils.isBlank(token)) {
            LOGGER.error("获取当前登录的token失败，token{}", token);
            return null;
        }

        String userId = JWT.decode(token).getAudience().get(0);
        return staticUserService.getUserById(Long.valueOf(userId));
    } catch (Exception e) {
        LOGGER.error("获取当前用户信息失败，token{}", token, e);
        return null;
    }
}
```

2. **头像上传**
   【vue3/src/views/backend/user/PersonInfo.vue】

```
// 自定义头像上传方法
const customUploadAvatar = async (options) => {
  try {
    const { file } = options;

    // 创建 FormData 对象
    const formData = new FormData();
    formData.append("file", file);

    // 设置自定义上传选项
```

```
    const uploadOptions = {
      headers: {
        token: localStorage.getItem("token") || "",
      },
      // 不进行JSON处理
      transformRequest: [(data) => data],
      // 自定义成功消息
      successMsg: "头像上传成功",
      // 自定义错误消息
      errorMsg: "头像上传失败",
      // 成功回调
      onSuccess: async (data) => {
        // 更新用户头像
        userForm.avatar = data;

        // 保存到后端
        await updateUserAvatar(data);

        // 通知上传成功
        options.onSuccess({ data });
      },
      // 错误回调
      onError: (error) => {
        console.error("头像上传错误:", error);
        options.onError(new Error(error.message || "上传失败"));
      },
    };

    // 发送上传请求
    await request.post("/file/upload/img", formData, uploadOptions);
  } catch (error) {
    options.onError(error);
    console.error("头像上传过程发生错误:", error);
  }
};

// 更新用户头像信息
const updateUserAvatar = async (avatarPath) => {
  try {
    await request.put(
      `/user/${userForm.id}`,
      { avatar: avatarPath },
      {
        showDefaultMsg: false,
        onSuccess: (data) => {
          // 更新本地用户信息
          const updatedUserInfo = { ...userStore.userInfo, avatar: avatarPath };
          userStore.updateUserInfo(updatedUserInfo);
        },
        onError: (error) => {
          console.error("头像信息保存失败", error);
          ElMessage.error("头像信息保存失败");
        },
      }
    );
  } catch (error) {
```

```
    console.error("头像信息保存失败", error);
    ElMessage.error("头像信息保存失败");
    throw error;
  }
};
```

2.5.4 功能实现细节

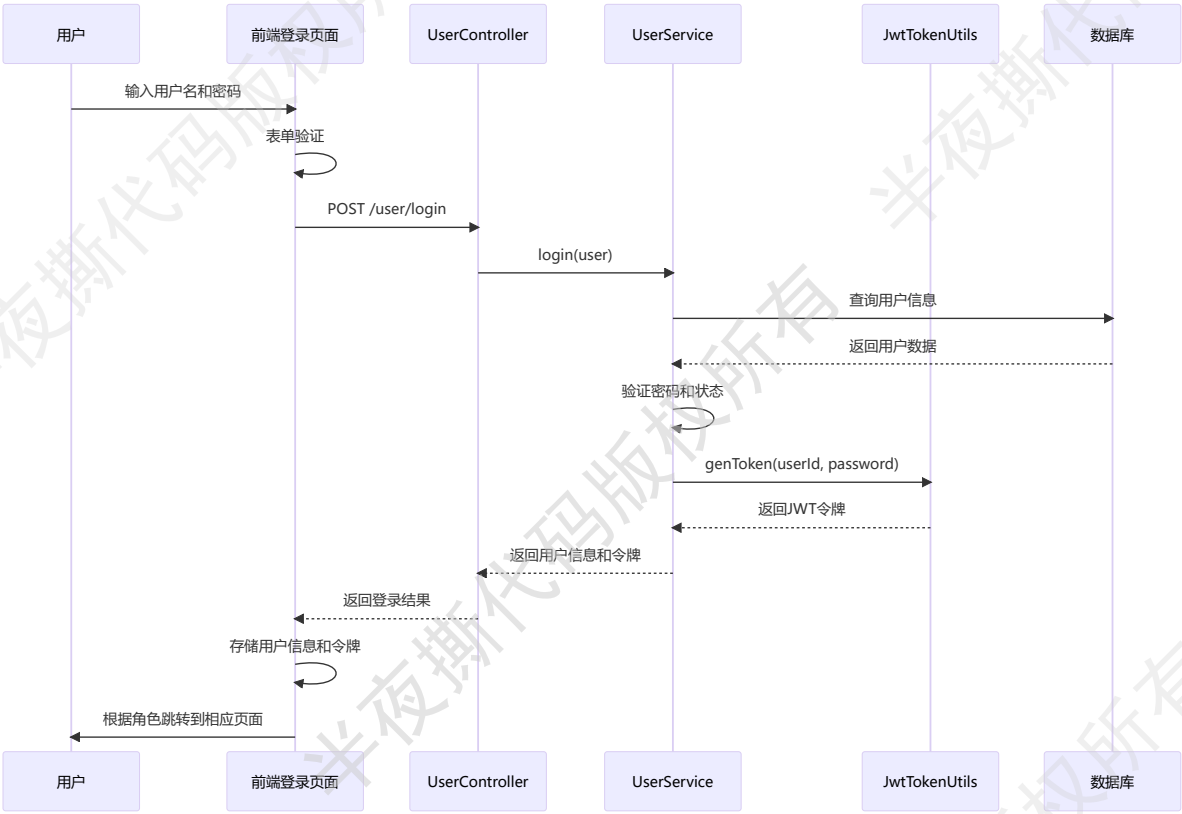
1. **JWT认证**：
   - 使用JWT令牌识别当前登录用户
   - 从请求头或请求参数中获取token
   - 解析token获取用户ID
   - 根据ID查询完整的用户信息
2. **文件上传**：
   - 使用FormData对象封装文件数据
   - 设置特殊的请求头和处理方式
   - 上传成功后更新用户头像信息
   - 同步更新前端状态和本地存储
3. **权限控制**：
   - 用户只能修改自己的信息
   - 通过JWT令牌确认用户身份
   - 后端验证操作权限

三、总结

3.1 用户登录功能实现

用户登录功能通过前端的handleLogin()方法实现，该方法首先进行表单验证，然后调用request.post("/user/login", loginForm)向后端发送登录请求。后端UserController的login方法接收请求并调用UserService的login方法进行处理。login方法首先通过getByUsername方法检查用户是否存在，然后验证账户状态是否正常，使用bCryptPasswordEncoder.matches()方法验证密码是否正确，最后通过JwtTokenUtils.genToken()生成JWT令牌并返回给前端。前端根据用户角色将用户信息存储在localStorage中并导航至相应页面（管理员/医生/护士进入后台，普通用户进入前台）。

半夜撕代码版权所有


用户
前端登录页面
UserController
UserService
JwtTokenUtils
数据库
输入用户名和密码
表单验证
POST /user/login
login(user)
查询用户信息
返回用户数据
验证密码和状态
genToken(userId, password)
返回JWT令牌
返回用户信息和令牌
返回登录结果
存储用户信息和令牌
根据角色跳转到相应页面
用户
前端登录页面
UserController
UserService
JwtTokenUtils
数据库

半夜撕代码版权所有

3.2 用户注册功能实现

用户注册功能通过前端的handleRegister()方法实现，该方法首先进行表单验证，然后调用request.post("/patient/register", registerData)向后端发送注册请求。后端PatientController的registerPatient方法接收请求，验证两次密码是否一致，然后创建用户和患者对象，调用patientService.createPatientWithUser方法进行处理。该方法首先检查用户名和邮箱唯一性，然后使用BCryptPasswordEncoder.encode()加密密码，设置默认状态和头像，最后保存用户和患者信息到数据库。注册成功后，前端显示成功消息并跳转到登录页面。

3.3 密码管理功能实现

密码管理功能包括密码修改和忘记密码两部分。密码修改通过前端的updatePassword()方法实现，该方法验证两次输入的新密码是否一致，然后调用request.put(`/user/password/${userInfo.id}`, passwordData)向后端发送修改请求。后端UserController的updatePassword方法接收请求并调用UserService的updatePassword方法进行处理，该方法首先验证旧密码是否正确，然后加密新密码并更新数据库。

忘记密码功能通过前端的handleForgetPassword()方法实现，该方法调用request.post("/user/forget", forgetForm)向后端发送重置请求。后端UserController的forgetPassword方法接收请求并调用UserService的forgetPassword方法进行处理，该方法验证邮箱是否存在，然后加密新密码并更新数据库。密码重置成功后，前端显示成功消息并跳转到登录页面。

3.4 用户管理功能实现

用户管理功能通过前端的UserList组件实现，包括用户列表展示、分页查询、条件筛选、新增用户、编辑用户信息、启用/禁用用户、删除用户等功能。用户列表查询通过fetchUsers()方法实现，该方法调用request.get('/user/page')向后端发送请求，后端UserController的getUsersByPage方法处理请求并返回分页数据。

新增和编辑用户通过submitForm()方法实现，该方法根据dialogType判断是新增还是编辑，然后调用相应的API向后端发送请求。后端UserController的createUser和updateUser方法处理请求并调用UserService的相应方法进行处理。删除用户通过handleDelete()方法实现，该方法调用request.delete(`/user/delete/${row.id}`)向后端发送请求，后端UserController的deleteUserById方法处理请求并调用UserService的deleteUserById方法进行处理，该方法首先检查用户是否存在和是否被关联，然后执行删除操作。

3.5 当前用户信息管理功能实现

当前用户信息管理功能允许已登录用户查看和修改自己的个人信息。获取当前用户信息通过JwtTokenUtils.getCurrentUser()方法实现，该方法从请求头或请求参数中获取token，解析token获取用户ID，然后查询完整的用户信息。更新用户信息通过前端的handleSave()方法实现，该方法调用request.put(`/user/${form.id}`, form)向后端发送请求，后端UserController的updateUser方法处理请求并调用UserService的updateUser方法进行处理。

头像上传通过customUploadAvatar()方法实现，该方法创建FormData对象并添加文件，然后调用request.post("/file/upload/img", formData)向后端发送请求，上传成功后调用updateUserAvatar()方法更新用户头像信息。

半夜撕代码版权所有